2026-7-7 Linux开发工具--代码调试器

- 1. 调试器 gdb 的定位
  - 1.1 调试器解决什么问题
    - 1.1.1 从“程序结果不对”说起
    - 1.1.2 调试的基本闭环
  - 1.2 debug 与 release
    - 1.2.1 两种常见构建模式
    - 1.2.2 为什么发布版本通常不是 debug
    - 1.2.3 软件开发流程中的 debug 与 release
  - 1.3 为什么默认构建往往无法调试
    - 1.3.1 gcc/g++ 默认不保留完整调试信息
    - 1.3.2 生成 debug 版本
    - 1.3.3 可执行文件与 ELF
  - 1.4 检查程序是否包含调试信息
    - 1.4.1 使用 readelf 查看 ELF 节区
    - 1.4.2 搜索 debug 节区
    - 1.4.3 使用 file 辅助判断
    - 1.4.4 重新编译后确认
- 2. 安装和启动 gdb
  - 2.1 安装调试工具
    - 2.1.1 Ubuntu/Debian
    - 2.1.2 CentOS/Red Hat
  - 2.2 启动 gdb
    - 2.2.1 打开 debug 程序
    - 2.2.2 没有调试信息时
  - 2.3 启动 cgdb
    - 2.3.1 gdb 与 cgdb 的区别
    - 2.3.2 cgdb 的屏幕切换
  - 2.4 退出 gdb
- 3. 查看源码与执行控制
  - 3.1 查看源码
    - 3.1.1 list 基本用法
    - 3.1.2 查看指定位置
    - 3.1.3 分屏查看源码
  - 3.2 运行程序
    - 3.2.1 run

- 3.3 三种常用执行方式
  - 3.3.1 next：逐过程
  - 3.3.2 step：逐语句
  - 3.3.3 continue：继续运行
- 3.4 until、finish 与区域跳转
  - 3.4.1 until
  - 3.4.2 finish
  - 3.4.3 三者如何帮助缩小范围
- 4. 断点的使用
  - 4.1 设置断点
    - 4.1.1 在行号设置
    - 4.1.2 在函数入口设置
    - 4.1.3 在指定文件设置
    - 4.1.4 设置后执行
  - 4.2 查看和管理断点
    - 4.2.1 查看断点列表
    - 4.2.2 删除断点
    - 4.2.3 临时禁用和启用
  - 4.3 一次运行到下一个断点
  - 4.4 断点与调试范围
    - 4.4.1 不要一开始到处打断点
    - 4.4.2 断点的判断依据
- 5. 变量、表达式与调用栈
  - 5.1 print 查看变量
    - 5.1.1 基本用法
    - 5.1.2 检查值的类型和信息
  - 5.2 display 持续监视
    - 5.2.1 开始监视
    - 5.2.2 查看编号
    - 5.2.3 取消显示
  - 5.3 查看调用栈
    - 5.3.1 backtrace
    - 5.3.2 查看当前栈帧局部变量
    - 5.3.3 切换栈帧
- 6. 观察变量变化
  - 6.1 watch
    - 6.1.1 基本用法

        - 6.1.2 为什么需要先进入变量作用域
        - 6.1.3 watch 与 display 的区别
    - 6.2 set var
        - 6.2.1 在运行中修改变量
        - 6.2.2 验证问题原因
    - 6.3 运行时修改不等于修复代码
- 7. 条件断点
    - 7.1 为什么要用条件断点
    - 7.2 新增条件断点的位置
        - 7.2.1 当前文件行号
        - 7.2.2 指定文件行号
        - 7.2.3 函数入口
    - 7.3 给已有断点追加条件
    - 7.4 条件断点的执行
    - 7.5 条件本身要谨慎
- 8. 一个完整调试示例
    - 8.1 示例程序
    - 8.2 构建 debug 版本
    - 8.3 在 results 计算处停下
    - 8.4 用 set var 验证
    - 8.5 修复源码
    - 8.6 使用 watch 捕获变化
    - 8.7 使用条件断点观察循环
- 9. 调试方法论
    - 9.1 先缩小问题范围
        - 9.1.1 用断点分区
        - 9.1.2 不要盲目逐行执行
    - 9.2 观察而不是猜测
        - 9.2.1 建立预期
        - 9.2.2 验证单一原因
    - 9.3 从运行时实验回到源码
    - 9.4 调试时应该记录什么
- 10. gdb 命令速查与练习
    - 10.1 启动与退出
    - 10.2 源码与运行
    - 10.3 断点
    - 10.4 单步与控制

- 10.5 变量与栈
- 10.6 条件断点
- 10.7 建议练习
  - 10.7.1 检查调试信息
  - 10.7.2 基础调试
  - 10.7.3 范围跳转
  - 10.7.4 条件断点
  - 10.7.5 watch 与运行时实验
- 10.8 调试学习主线

1. 调试器 gdb 的定位

1.1 调试器解决什么问题

1.1.1 从“程序结果不对”说起

程序能编译、能启动，不代表逻辑一定正确。常见问题包括：

- 程序崩溃。
- 结果和预期不符。
- 某个变量突然变成错误值。
- 循环在错误的次数停止。
- 函数没有被正确调用。
- 某段代码根本没有执行。

调试器的任务不是替程序员“想明白错误”，而是帮助程序员观察程序运行状态、缩小怀疑范围，从而找到问题所在的代码和原因。

1.1.2 调试的基本闭环

```
发现问题
→ 复现问题
→ 缩小代码范围
→ 观察变量和调用过程
→ 提出原因假设
→ 验证假设
→ 修改源码
→ 重新构建并回归测试
```

真正分析问题的人仍然是程序员。gdb 提供的是观察、暂停、跳转和检查状态的工具。

1.2 debug 与 release

1.2.1 两种常见构建模式

模式	主要用途	特点
debug	开发和定位问题	包含源码行号、变量类型等调试信息，文件通常更大
release	交付和发布	更关注体积和优化，用户通常不需要调试信息

Debug 版本中的调试信息主要给开发工具使用，对最终用户没有直接价值。

1.2.2 为什么发布版本通常不是 debug

调试信息会增加可执行文件体积。正式发布时，软件更关注：

- 文件大小。
- 运行效率。
- 不暴露内部符号和实现细节。

因此开发阶段使用 debug，测试和发布阶段通常使用 release。

1.2.3 软件开发流程中的 debug 与 release

课堂中用一条简化流程说明二者的位置：

```
调研和需求
→ 立项和资源准备
→ 开发功能
→ 自测与调试
→ 提交代码
→ 审核与合并
→ 联合测试
→ 发布 release
```

开发阶段需要 debug 版本帮助定位问题；最终交付给用户的通常是 release 版本。

1.3 为什么默认构建往往无法调试

1.3.1 gcc/g++ 默认不保留完整调试信息

直接执行：

```
gcc mycmd.c -o mycmd
```

gcc 默认进行普通构建，通常不会加入完整调试符号。因此 gdb 打开程序时可能提示：

```
(no debugging symbols found)
```

这并不表示程序不能运行，只表示调试器拿不到完整的源码和变量信息。

1.3.2 生成 debug 版本

开发阶段推荐：

```
gcc -g -O0 mycmd.c -o mycmd-debug
```

- `-g`：生成调试信息。
- `-O0`：基本关闭优化，减少优化对单步和变量观察的干扰。

也可以写成：

```
gcc mycmd.c -o mycmd-debug -g
```

1.3.3 可执行文件与 ELF

Linux 可执行文件不仅仅是机器码的集合，它们通常遵循 ELF（Executable and Linkable Format）格式。

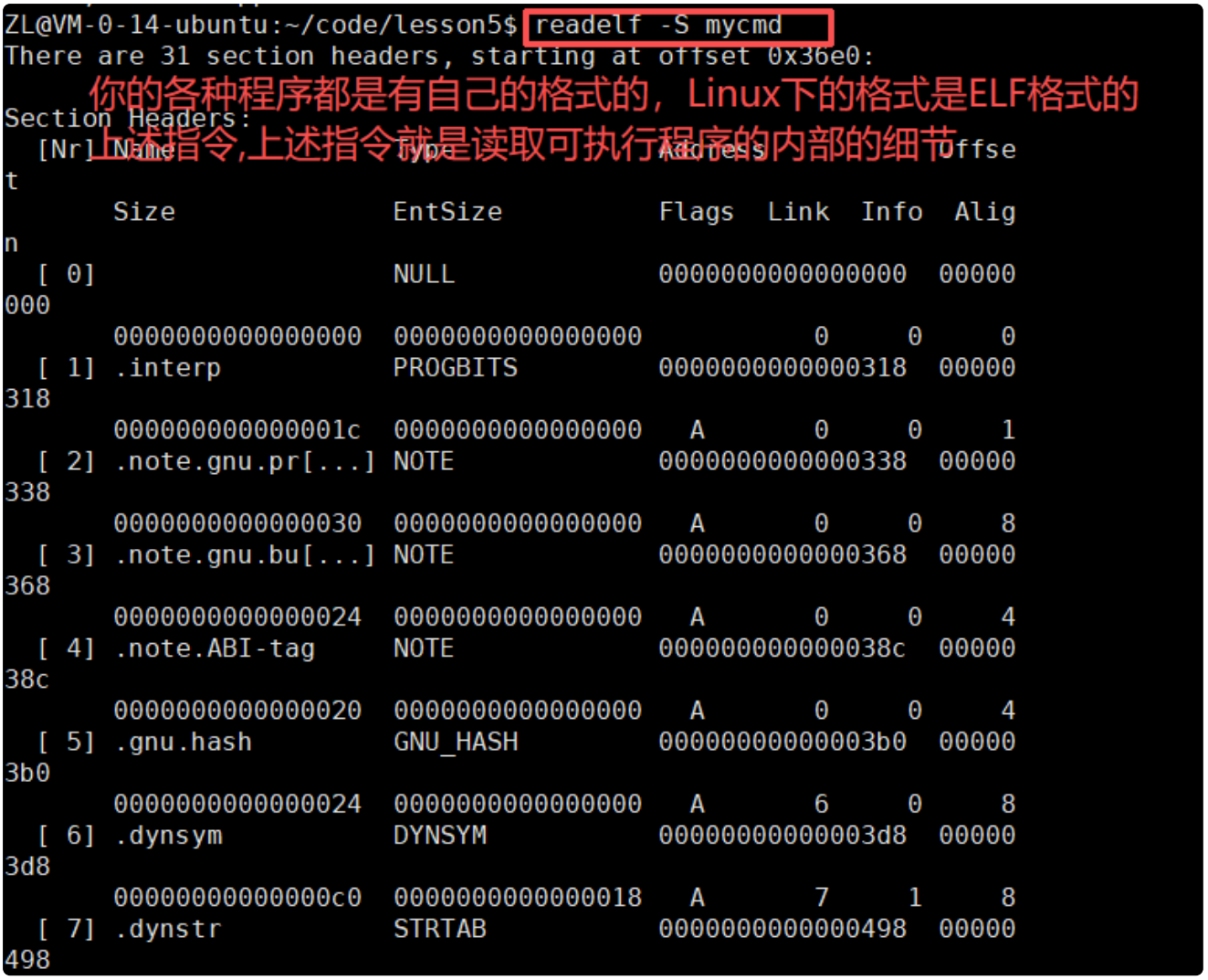
ELF 中包含代码、数据、符号表、节区等结构，调试信息也以特定节区形式存在。

1.4 检查程序是否包含调试信息

1.4.1 使用 readelf 查看 ELF 节区

```
readelf -S mycmd-debug
```

`readelf -S` 显示 ELF 的节区表，可用来观察程序内部格式信息。



1.4.2 搜索 debug 节区

```
readelf -S mycmd-debug | grep -i debug
```

如果包含调试信息，常见输出包括：

```
.debug_info
.debug_abbrev
.debug_line
.debug_str
```

```
ZL@VM-0-14-ubuntu:~/code/lesson5$ readelf -S mycmd-debug | grep -i debug
  [28] .debug_aranges    PROGBITS         0000000000000000  0000303d
  [29] .debug_info       PROGBITS         0000000000000000  0000306d
  [30] .debug_abbrev     PROGBITS         0000000000000000  000031a8
  [31] .debug_line       PROGBITS         0000000000000000  0000327c
  [32] .debug_str        PROGBITS         0000000000000000  00003306
  [33] .debug_line_str   PROGBITS         0000000000000000  000033f3
ZL@VM-0-14-ubuntu:~/code/lesson5$
```

如果普通构建执行相同命令后没有输出，说明没有这些调试节区。

1.4.3 使用 file 辅助判断

```
file mycmd
file mycmd-debug
```

`file` 可以显示：

- 文件类型。
- CPU 架构。
- 是否动态链接。
- 是否 stripped。

`file` 输出不能完全替代 `readelf` 的调试节区检查，但可以快速确认程序的基本格式。

1.4.4 重新编译后确认

先构建普通版本和 debug 版本：

```
gcc mycmd.c -o mycmd
gcc mycmd.c -o mycmd-debug -g
```

再分别检查：

```
readelf -S mycmd | grep -i debug
readelf -S mycmd-debug | grep -i debug
```

```
[28] .debug_aranges    PROGBITS         0000000000000000  0000303d
     0000000000000030  0000000000000000           0     0     1
[29] .debug_info       PROGBITS         0000000000000000  0000306d
     000000000000013b  0000000000000000           0     0     1
[30] .debug_abbrev     PROGBITS         0000000000000000  000031a8
     00000000000000d4  0000000000000000           0     0     1
[31] .debug_line       PROGBITS         0000000000000000  0000327c
     000000000000008a  0000000000000000           0     0     1
[32] .debug_str        PROGBITS         0000000000000000  00003306
     00000000000000ed  0000000000000001  MS       0     0     1
[33] .debug_line_str   PROGBITS         0000000000000000  000033f3
     0000000000000033  0000000000000001  MS       0     0     1
```

debug 版本应当出现 `.debug_*` 节区。

2. 安装和启动 gdb

2.1 安装调试工具

2.1.1 Ubuntu/Debian

```
sudo apt update
sudo apt install -y gdb cgdb
```

2.1.2 CentOS/Red Hat

```
sudo yum install -y gdb cgdb
```

如果只需要纯命令行调试器，可以只安装 gdb；cgdb 是在 gdb 基础上提供源码分屏界面的前端工具。

2.2 启动 gdb

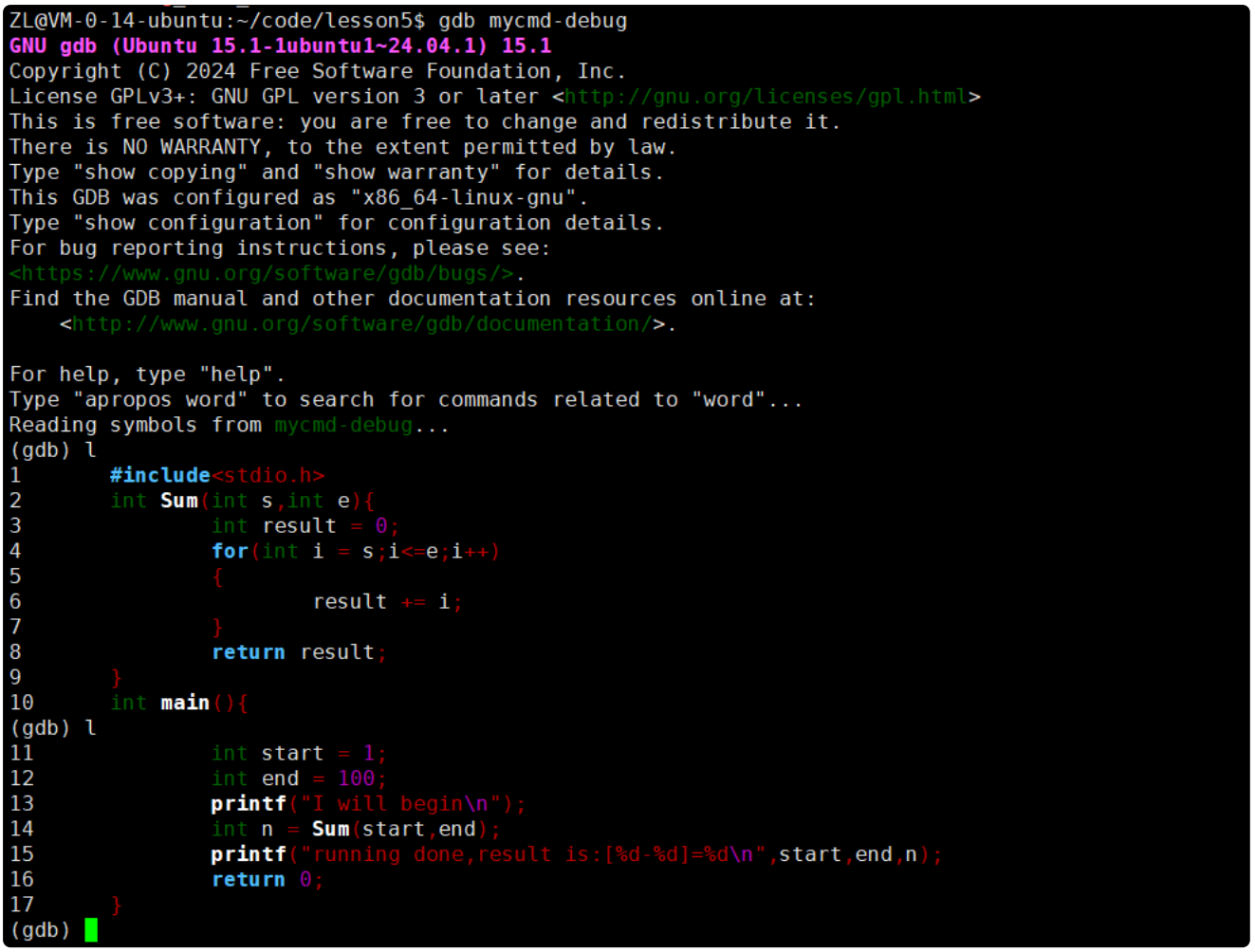
2.2.1 打开 debug 程序

```
gdb ./mycmd-debug
```

进入 gdb 后会出现：

```
(gdb)
```

提示符表示 gdb 正在等待调试命令。



2.2.2 没有调试信息时

如果直接打开 release 程序：

```
gdb ./mycmd
```

通常会看到：

```
Reading symbols from ./mycmd...
(no debugging symbols found)
```

此时仍可执行部分底层调试，但无法正常查看源码行号和完整变量类型，学习阶段应使用 `-g` 构建。

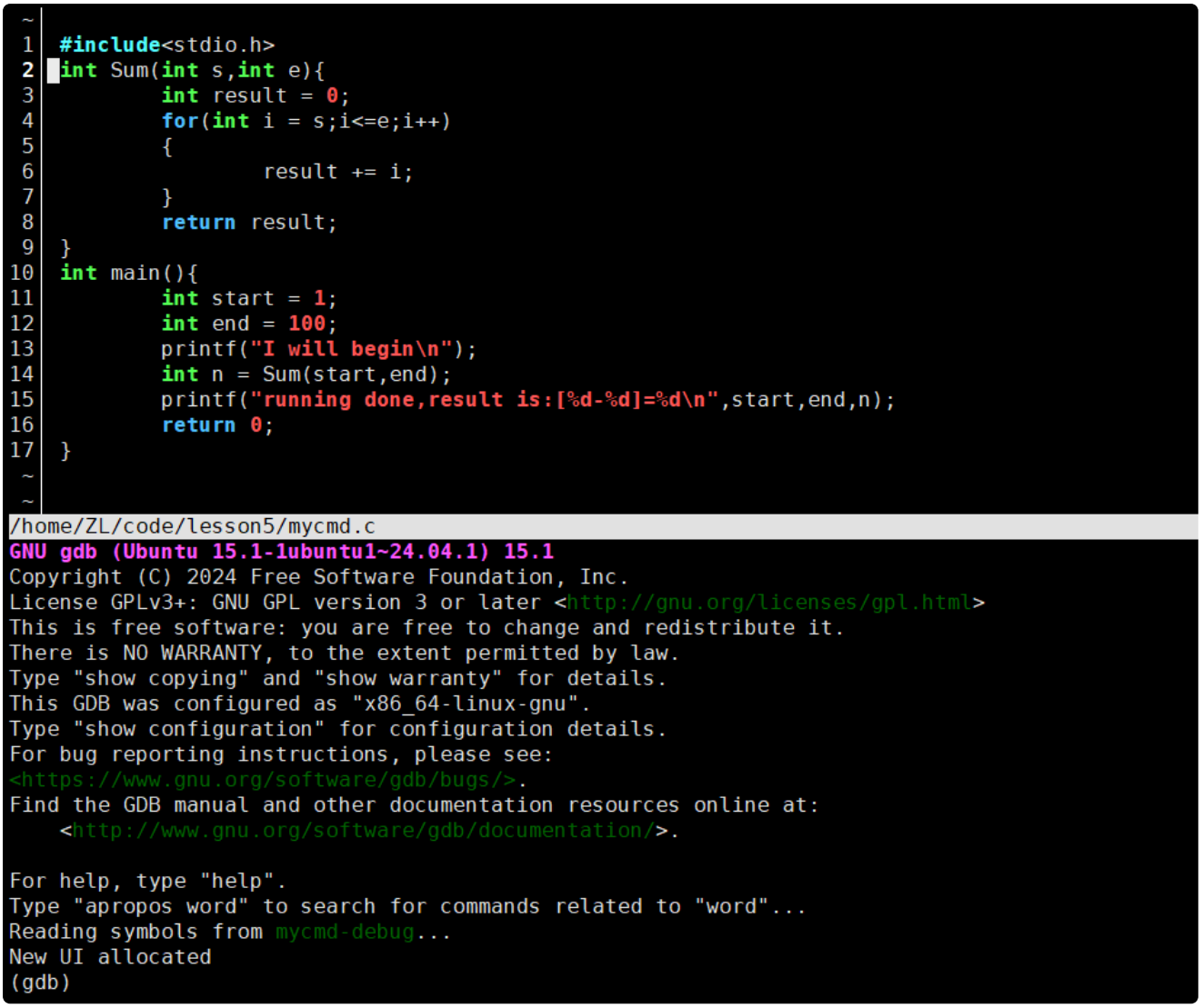
2.3 启动 cgdb

2.3.1 gdb 与 cgdb 的区别

```
gdb  = 调试核心和命令解释器
cgdb = 在 gdb 外增加源码分屏显示
```

cgdb 底层仍使用 gdb，调试命令基本一致。

```
cgdb ./mycmd-debug
```



2.3.2 cgdb 的屏幕切换

- 按 `Esc`：进入上方的源码屏。
- 按 `i`：回到下方的 gdb 命令屏。

- 在源码屏中可以用方向键浏览代码。

2.4 退出 gdb

```
quit
```

或者按 `Ctrl+D`。

如果程序仍在运行，gdb 可能询问是否终止调试进程。

3. 查看源码与执行控制

3.1 查看源码

3.1.1 list 基本用法

```
list
l
```

通常从当前位置开始显示 10 行源码。连续执行 `l` 会继续向下显示。

3.1.2 查看指定位置

```
l 20
l mycmd.c:10
l main
```

分别表示：

命令	作用
`l 20`	查看当前文件第 20 行附近
`l mycmd.c:10`	查看指定文件第 10 行附近
`l main`	查看 main 函数附近

3.1.3 分屏查看源码

纯 gdb 可以使用：

```
layout src
```

显示源码窗口。常见 TUI 操作：

```
layout src
layout asm
layout regs
```

如果界面显示混乱，可以按 `Ctrl+L` 重绘。

3.2 运行程序

3.2.1 run

```
run
r
```

从程序开始执行。如果设置了断点，程序会在第一个命中的断点处停下。

如果程序需要参数，可以直接写在 `run` 后面：

```
r arg1 arg2
```

3.3 三种常用执行方式

3.3.1 next：逐过程

```
next
n
```

执行当前行，并把函数调用当作一个整体，不进入函数内部。类似许多 IDE 中的 F10。

适合场景：

- 已经确认函数内部实现正确。
- 只关心函数返回后的结果。

3.3.2 step：逐语句

```
step
s
```

如果当前行调用了函数，会进入函数内部继续执行。类似 F11。

适合场景：

- 怀疑函数内部逻辑。
- 需要观察实参进入函数后的变化。

3.3.3 continue：继续运行

```
continue
c
```

从当前位置继续执行，直到：

- 命中下一个断点。
- 程序结束。
- 程序收到信号并停止。

3.4 until、finish 与区域跳转

3.4.1 until

```
until 行号
until 20
```

在当前函数内执行到指定行，常用于跳过已经确认正常的循环或代码段。

3.4.2 finish

```
finish
```

执行完当前函数并返回调用位置，通常会显示函数返回值。

3.4.3 三者如何帮助缩小范围

```
continue → 跳到下一个怀疑位置
until    → 在函数内跳到指定位置
finish   → 执行完整个函数并检查返回值
```

这些命令可以减少无意义单步，把注意力集中到真正可疑的区域。

4. 断点的使用

4.1 设置断点

4.1.1 在行号设置

```
break 20
b 20
```

表示在当前源码文件的第 20 行设置断点。

4.1.2 在函数入口设置

```
b main
b Sum
```

4.1.3 在指定文件设置

```
b mycmd.c:10
b other.c:25
```

当工程有多个源文件时，必须明确指定文件名。

4.1.4 设置后执行

```
r
```

程序运行到断点后暂停：

```
Breakpoint 1, main () at mycmd.c:20
20    int n = Sum(start, end);
```

此时可以查看变量、源码和调用栈。

4.2 查看和管理断点

4.2.1 查看断点列表

```
info breakpoints
info b
```

会显示：

- 断点编号。
- 类型。
- 是否启用。
- 地址。
- 所在文件和行号。
- 已命中次数。

4.2.2 删除断点

删除指定断点：

```
delete 1
```

删除全部断点时 gdb 通常会要求确认：

```
delete breakpoints
```

4.2.3 临时禁用和启用

```
disable breakpoints
enable breakpoints
```

禁用不会删除断点，之后仍可重新启用。

4.3 一次运行到下一个断点

在多个断点之间调试时，频繁单步效率很低。可以：

```
c
```

让程序直接运行到下一个断点。

4.4 断点与调试范围

4.4.1 不要一开始到处打断点

断点过多会让程序频繁暂停，反而难以分析。更有效的方式是：

```
先确认错误的大致阶段
→ 在阶段边界设置少量断点
→ 逐步缩小范围
```

4.4.2 断点的判断依据

设置断点前应先问：

- 这个位置真的可能产生错误吗?
- 如果程序到达这里，预期变量应该是什么?
- 如果这里是正确的，错误在之前还是之后?

5. 变量、表达式与调用栈

5.1 print 查看变量

5.1.1 基本用法

```
print 变量
p 变量
p 表达式
```

例子:

```
p result
p start + end
p flag
```

`print` 类似调试过程中的 `printf`，但不需要重新修改源码和编译。

5.1.2 检查值的类型和信息

```
ptype 变量
p/x 变量
p/d 变量
```

分别查看类型、十六进制值和十进制值。

5.2 display 持续监视

5.2.1 开始监视

```
display i
```

```
display result
```

程序每次停下时都会自动显示对应值。

5.2.2 查看编号

使用：

```
info display
```

查看每个持续显示项对应的编号。

5.2.3 取消显示

```
undisplay 1
```

取消编号为 1 的持续显示项。

5.3 查看调用栈

5.3.1 backtrace

```
backtrace
bt
```

显示当前调用栈，包括各个函数、参数和源码位置。

例如可以判断：

```
main
  → Sum
    → helper
```

当前实际执行点在哪个函数中，以及它是如何被调用到这里的。

5.3.2 查看当前栈帧局部变量

```
info locals
```

只查看当前函数中的局部变量。

5.3.3 切换栈帧

如果希望查看调用链中另一层函数的变量，可以在 `bt` 显示的编号基础上切换：

```
frame 1
```

然后再执行 `p` 或 `info locals`。

6. 观察变量变化

6.1 watch

6.1.1 基本用法

```
watch 变量
```

程序运行期间，如果被监视变量发生变化，gdb 会暂停并显示旧值与新值。

适合场景：

- 某个变量不应该被修改，却发生了修改。
- 指针突然变成非法地址。
- 计数器在错误位置被修改。
- 状态标志意外改变。

6.1.2 为什么需要先进入变量作用域

局部变量只有进入其所在函数后才会存在。通常需要先在函数内部停下，再执行：

```
watch result
```

6.1.3 watch 与 display 的区别

命令	行为
`display`	每次停下时显示当前值
`watch`	变量值变化时主动暂停程序

`watch` 更适合捕获“什么时候被改坏”，`display` 更适合观察连续变化。

6.2 set var

6.2.1 在运行中修改变量

```
set var flag = 1
```

它只改变当前调试进程中的值，不会修改源代码。

6.2.2 验证问题原因

如果怀疑某个标志导致结果错误，可以先在 gdb 中改正该值，观察程序输出是否恢复正常。

```
p flag
set var flag = 1
p flag
n
```

如果结果立刻正确，就可以进一步检查源码中为什么 `flag` 初始值或执行路径不对。

6.3 运行时修改不等于修复代码

`set var` 是一种实验手段。关闭调试器后，源码中的错误仍然存在。

正确流程是：

```
通过 set var 验证原因
→ 定位对应的源码位置
→ 修改源码
→ 重新编译
→ 回归测试
```

7. 条件断点

7.1 为什么要用条件断点

在循环中，如果程序执行 1000 次，只希望在第 30 次停下，每次都手动单步效率很低。

条件断点可以写成：

```
b 9 if i == 30
```

表示第 9 行只有在 `i == 30` 时才停下。

7.2 新增条件断点的位置

7.2.1 当前文件行号

```
b 9 if i == 30
```

7.2.2 指定文件行号

```
b mycmd.c:9 if i == 30
```

7.2.3 函数入口

```
b Sum if s == 1
```

7.3 给已有断点追加条件

先查看断点编号：

```
info b
```

假设要修改 2 号断点：

```
condition 2 i == 30
```

注意语法差异：

```
新增断点：b 9 if i == 30
已有断点：condition 2 i == 30
```

`condition` 命令中不需要再写 `if`。

7.4 条件断点的执行

设置后执行：

```
c
```

程序会运行到条件成立时停下，并显示当前行和变量值。

7.5 条件本身要谨慎

如果条件表达式写错，可能永远不命中，或者引用当前作用域不存在的变量。

设置后应先用：

```
info b
```

确认条件已经正确附加。

8. 一个完整调试示例

8.1 示例程序

```c
#include <stdio.h>

int flag = 0;

int Sum(int start, int end)
{
    int result = 0;

    for (int i = start; i <= end; ++i)
        result += i;

    return result * flag;
}

int main(void)
{
    int start = 1;
    int end = 100;

    printf("I will begin\n");
    int n = Sum(start, end);
    printf("result = %d\n", n);

    return 0;
}
```

程序可能输出：

```
I will begin
result = 0
```

预期结果是 `5050`，但实际得到 `0`。

8.2 构建 debug 版本

```
gcc -g -O0 mycmd.c -o mycmd-debug
```

8.3 在 results 计算处停下

```
gdb ./mycmd-debug
l main
b 21
r
```

进入 `Sum`：

```
s
until 12
```

查看求和结果：

```
p result
```

如果 `result` 是 `5050`，说明循环求和本身正常；再检查：

```
p flag
```

发现：

```
$2 = 0
```

于是怀疑返回值被乘上了错误的标志。

8.4 用 set var 验证

```
set var flag = 1
n
c
```

继续执行后输出变成 `5050`，说明 `flag` 的值就是问题原因。

8.5 修复源码

运行时修改变量只能验证原因，最终需要修改源码中的初始值或逻辑，再重新编译。

8.6 使用 watch 捕获变化

如果变量不是一开始就错，而是运行中被某段代码改错，可以进入对应作用域后：

```
watch flag
c
```

gdb 会在变量变化时暂停，并显示旧值和新值。

8.7 使用条件断点观察循环

如果只想观察第 30 次循环：

```
b 9 if i == 30
c
p i
p result
```

这样可以避免手动执行前面 29 次循环。

9. 调试方法论

9.1 先缩小问题范围

9.1.1 用断点分区

在可能的阶段边界设置断点：

```
输入是否正确
→ 中间计算是否正确
→ 输出是否正确
```

如果某一段前面正确、后面错误，就可以把怀疑范围缩小到这一段。

9.1.2 不要盲目逐行执行

对大函数逐步执行代价很高。应先用：

- `finish` 跳过已经确认正常的函数。
- `until` 跳过正常循环。
- `continue` 跳到下一个关键断点。

9.2 观察而不是猜测

9.2.1 建立预期

在每个关键位置回答：

```
这里的变量本来应该是什么？
现在实际是什么？
从什么时候开始不一致？
```

9.2.2 验证单一原因

一次只改变一个条件或变量，观察结果是否按预期变化。否则很难判断是哪一个修改起了作用。

9.3 从运行时实验回到源码

调试器的操作不是最终修复：

```
发现现象
→ 用 gdb 验证原因
→ 回到源码修改
→ 重新构建
→ 再次验证
```

9.4 调试时应该记录什么

建议记录：

- 使用的输入。
- 预期输出。
- 实际输出。
- 停下的函数与行号。
- 关键变量的值。
- 改变某个变量后的结果。
- 最终确认的原因。

这些信息可以帮助复现问题，也可以避免以后重复踩同样的坑。

10. gdb 命令速查与练习

10.1 启动与退出

```
gdb ./mycmd-debug
cgdb ./mycmd-debug
quit
Ctrl+D
```

10.2 源码与运行

```
list
l
l 20
l mycmd.c:10
l main
layout src
run
r
```

10.3 断点

```
break 20
b 20
b main
b mycmd.c:10
info breakpoints
info b
delete 1
```

```
disable breakpoints
enable breakpoints
```

10.4 单步与控制

```
next
n
step
s
continue
c
until 20
finish
```

命令	含义	常见快捷键类比
`next`	逐过程，不进入函数	F10
`step`	逐语句，进入函数	F11
`continue`	运行到下一个停止点	F5 的继续
`until`	执行到指定行	跳到范围内目标
`finish`	执行到当前函数返回	跳出当前函数

10.5 变量与栈

```
print result
p result
p start + end
ptype result
display i
info display
undisplay 1
info locals
backtrace
bt
frame 1
set var flag = 1
watch flag
```

10.6 条件断点

```
b 9 if i == 30
b mycmd.c:9 if i == 30
b Sum if i == 30
condition 2 i == 30
info b
c
```

10.7 建议练习

10.7.1 检查调试信息

1. 用普通方式编译程序。
2. 执行 `readelf -S mycmd | grep -i debug`。
3. 用 `-g` 重新编译。
4. 再检查一次并比较结果。
5. 用 `file` 比较两个程序的格式信息。

10.7.2 基础调试

1. 在 `main` 函数设置断点。
2. 使用 `r` 运行到断点。
3. 使用 `n` 逐过程执行。
4. 使用 `s` 进入函数。
5. 观察 `p`、`display` 和 `info locals` 的输出。

10.7.3 范围跳转

1. 在循环结束后设置断点。
2. 使用 `until` 跳过循环中已确认正常的部分。
3. 使用 `finish` 跳出函数并查看返回值。
4. 使用 `c` 跳到下一个断点。

10.7.4 条件断点

1. 在循环中设置只在 `i == 30` 时停止的断点。
2. 用 `info b` 检查条件。
3. 用 `c` 运行到条件成立。

4. 查看循环变量和累计结果。

10.7.5 watch 与运行时实验

1. 选择一个不应该改变的变量。
2. 使用 `watch` 捕获修改位置。
3. 使用 `set var` 临时修正值，验证是否影响输出。
4. 回到源码真正修复问题。

10.8 调试学习主线

```
区分 debug 与 release
→ 用 -g 构建可调试程序
→ 用 readelf 确认调试信息
→ 在关键位置设置断点
→ 用单步和范围跳转缩小问题
→ 用 print/display/watch 观察状态
→ 用 set var 和条件断点验证假设
→ 回到源码修复并重新测试
```

本文只整理 gdb/cgdb 调试相关内容，不展开编译器原理、Makefile、进度条、Git 或其他开发工具。